

Forecasting Turning Points in Stock Price by Integrating Chart Similarity and Multipersistance

Shangzhe Li[✉], Yingke Liu, Xueyuan Chen, Junran Wu[✉], and Ke Xu

Abstract—Forecasting financial data plays a crucial role in financial market. Relying solely on prices or price trends as prediction targets often leads to a vast of invalid transactions. As a result, researchers have increasingly turned their attention to turning points as the prediction target. Surprisingly, existing methods have largely overlooked the role of technical charts, despite turning points being closely related to the technical charts. Recently, several researchers have attempted to utilize chart information via converting price sequences into images for turning point forecasting, but robustness and convergence problems arise. To address these challenges and enhance the turning point predictions, this article introduces a new method known as MPCNet. Specifically, we first transform the price series into a graph structure using chart similarity to robustly extract valuable information from technical charts. Additionally, we introduce the multipersistance topology tool to accurately predict stock turning points and provide convergence guarantee. Experimental results demonstrate the significant superiority of our proposed model over existing methods. Furthermore, based on additional performance evaluations using real stock data, MPCNet consistently achieves the highest average return during the transaction backtesting period. Meanwhile, we provide empirical validation of robustness and theoretical analysis to confirm its convergence, establishing it as a superior tool for financial forecasting.

Index Terms—Chart similarity, multipersistance, turning point forecasting.

I. INTRODUCTION

PREDICTING the stock market is always a pressing concern for most investors. However, targeting prices or price trends alone can often result in excessive and ineffective trading, accompanied by high transaction fees [10]. Consequently, researchers gradually shift their focus towards predicting turning points (TPs) that allow for more timely and strategic trading decisions. A TP occurs when a previously upward or downward trend reverses, leading to a subsequent decline or increase over a specific period of time [12], [51]. These TPs embody crucial

market information as they signify distinctive shifts in system activity [41], thereby playing a vital role in generating profitable trades.

Currently, various methods have been proposed to identify TPs based on the provided definition [12], [51]. Traditional methods usually utilize leading indicators and probit models to identify TPs, but the significant proportion difference between TPs and non-turning points (OPs) could generally deteriorate the prediction performance of the model [41]. Fortunately, probabilistic analyses of order patterns [39], [42] and machine learning techniques [33] provide alternative methods for estimating equilibrium proportions of TPs. However, these methods primarily focus on price or technical indicators as input features of models, overlooking the close correlation between TPs and technical charts [44], [45], [46]. Khodaei et al. [23] attempted to address this issue by converting prices into images to identify TPs using chart information. Nevertheless, this approach faces two significant challenges. First, the converted images lack robustness, as small variations in pixel values can lead to drastic changes in image categories [40]. Second, convergence cannot be ensured when transforming prices into images, as the topological relationships among price points are disrupted. Therefore, there is a need for a robust and convergent method that leverages chart information to predict TPs accurately.

In current research on financial time series [30], [52], the newly emerged chart similarity approach, which is composed of Perceptual Important Point (PIP) [32] and Visual Graph (VG) algorithms [27], ensures robustness by utilizing chart information and transforming price sequences into graph structures [50]. Therefore, in light of the robustness of chart similarity, we aim to identify TPs through leveraging chart information. Moreover, we could further achieve the sample balance between TPs and OPs using the PIP algorithm [23], [41]. On the other hand, to ensure the convergence of our proposed method, we introduce multipersistance [5], [6], [7]. Recent research on integrating the shape characteristics of complex data into deep learning models has shown that using multipersistance to obtain topological representations can provide valuable insights for system organization, and enhance the convergence in graph learning mechanism [5], [6], [7]. Through the shape of charts in multipersistance, we roughly understand the invariant data attributes under continuous deformation (such as stretching, bending, and twisting) to better identify noise and effective information, which helps ensure convergence. As shown in the Fig. 1, only the last point of the two sequences is different. The upper sequence F point is the TP, while the lower sequence F point is not the

Manuscript received 1 September 2023; revised 20 May 2024; accepted 6 August 2024. Date of publication 16 August 2024; date of current version 13 November 2024. This work was supported in part by the Guangxi Science and Technology Major Project under Grant AA22067070 and in part by the Beijing Municipality and University Jointly Sponsored Research Training Program. Recommended for acceptance by M. A. Cheema. (Shangzhe Li and Yingke Liu contributed equally to this work.) (Corresponding author: Junran Wu.)

Shangzhe Li and Yingke Liu are with the School of Statistics and Mathematics, Central University of Finance and Economics, Beijing 102206, China.

Xueyuan Chen, Junran Wu, and Ke Xu are with the State Key Laboratory of Software Development Environment, Beihang University, Beijing 100191, China (e-mail: wu_junran@buaa.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2024.3444814>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2024.3444814

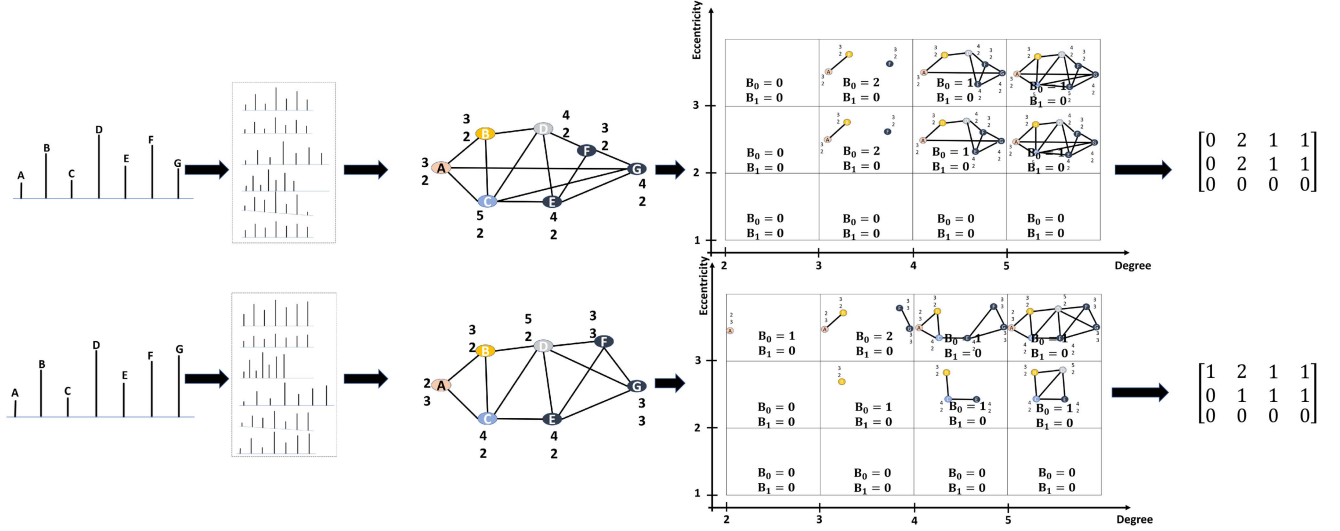


Fig. 1. An example of the robustness and convergence of our method. In the first and second panels from left, we transform two sequences under different perturbations into two corresponding graphs by the chart similarity combining PIP and VG. In the third, fourth and fifth panels from left, we demonstrate the process of calculating a transformed graph into a multipersistence matrix (Euler-Poincaré Surface). In the original graph (the third panel from left), the bottom numbers of nodes represent the eccentricity of the node, while the top numbers of nodes are the node degrees. The fifth panel from left depicts the corresponding Euler-Poincaré Surface. Shape properties of the original graph (the third panel from left) are evaluated along two geometric dimensions (i.e., degree and eccentricity). In particular, for each row, the corresponding graph is filtered by degree function. For each column, the corresponding graph is filtered by eccentricity function. Each cell includes Betti numbers B_0 and B_1 . Since B_1 is 0 for each cell, we get $\mathbb{E}P_{ij} = B_0$.

TP. We perform six types of perturbations simultaneously on two sequences, which are the most common types of perturbations in stock price sequences [30]. Finally, after passing through chart similarity and multipersistence, the perturbed sequences of the same original sequence are transformed into the same matrix under different perturbations, while the matrices generated by the perturbed sequences of different original sequences are different. The example suggests that our method which combines chart similarity and multipersistence can extract chart information about TPs while ensuring robustness and convergence. To further demonstrate the convergence of our method, we also provided theoretical proof of convergence in the Section III.

Given the superiority of chart similarity and multipersistence introduced above, we present our method, called MPCNet (MultiPersistence Chart graph neural Network), to predict TPs. Specifically, we first convert time series data into temporal graph data with chart similarity, while marking and expanding TPs using the developed PIP algorithm, which ensures the robustness of our method and the balance of sample proportion. After obtaining the transformed graph sequence, we introduce multipersistence to capture the inherent temporal conditional topological properties in the graph, ensuring the convergence of the method. Additionally, we present a spatio-temporal feature transformation method to effectively represent sequences of time-varying objects. To address the complex nature of high-dimensional data, we devise a spatial graph convolution module that facilitates the concurrent learning of co-evolving intra- and inter-dependencies, encompassing both spatial and temporal correlations. Compared with several state-of-the-art baselines, our framework has reached the best prediction accuracy of TPs on CSI-300 and SZ-50 components in 2020: 70.87% and 74.16%, respectively. In the trade simulation, our model obtains

the highest returns of 1.3 and 1.38 in 2020. Finally, we conduct discussions to explore the impact of different filters and the advantages of the trading signals generated by our model.

To summarize, the main contributions of this paper are as follows:

- In this study, we propose a novel approach called Multipersistence Chart Graph Convolutional Network (MPCNet) to predict TPs through the perspective of technical charts, which incorporates multipersistence and chart similarity.
- We have conducted empirical validation to confirm the robustness of MPCNet and have provided a theoretical analysis to demonstrate its convergence.
- Our method outperforms baselines in forecast accuracy and obtains the largest excess return on real stock data.

The remainder of this paper is structured as follows: Section II presents relevant works and research. The suggested technique is then thoroughly explained in Section III, which is followed by the introduction of numerous used variables. The experimental circumstances are described in Section IV. Section V presents the results. We discuss the results in Section VI, and a conclusion is offered in Section VII.

II. RELATED WORK

A. Turning Point Prediction

In order to develop successful strategies, it is crucial to identify the TPs of the stock price trend [10]. Probit models and other leading indicators are frequently used to identify TPs [4], [17], [25]. Moving average convergence/divergence indicators and the momentum index are two of the current leading indicators that are often utilized to identify TPs in the financial market [4]. A more complex indicator, such as the time-dependent Hurst

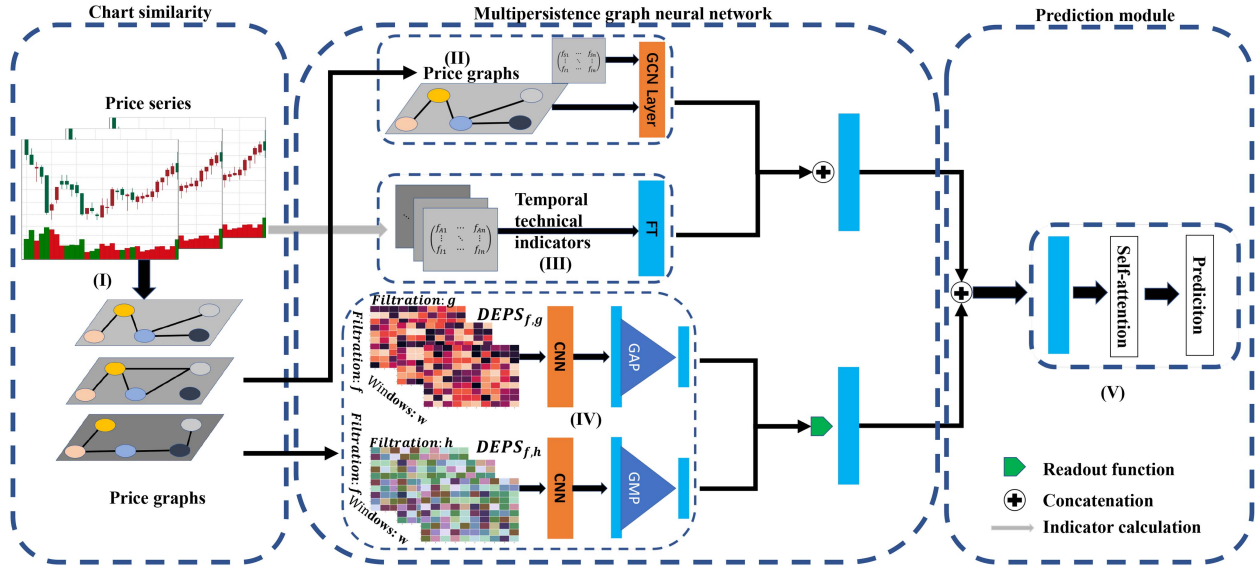


Fig. 2. The framework of our method. (I) Price graph generation via PIP and VG. (II) The spatial graph convolutional layer operates on $\mathcal{G}_t$ and captures spatial information at time t , as shown in (16). (III) The feature transformation (FT) module leverages a sliding window of size T to learn representations of the spatio-temporal data $\mathbf{X}_T$, as described in (15). (IV) The MEPSRL module is composed of a CNN base model and a global pooling mechanism, as depicted in (17) and (18). It enables the learning of MEPS with various types of multifiltrations, facilitating the understanding of the intrinsic graph structure across multiple geometric dimensions and time. (V) The learned spatio-temporal dependencies are then fed into an attention layer, as illustrated in (19) and (20), for TP forecasting.

index, is a gauge of market fractal properties that, in certain circumstances, can be utilized to identify possible TPs [17], [25]. However, these methods are often limited by the proportion imbalance problem of TPs, which is generally manifested [41].

Many researchers solve the problem of sample imbalance through probability analysis of order patterns [18], [39], [42] and machine learning methods [29], [33], [53]. Piecewise linear representation (PLR) method is the most commonly used algorithm to expand the TPs. On this basis, support vector machine (SVM) [41], Bayesian graph piecewise vector autoregression [1] and nonlinear dynamic system model [10] are employed to predict the expanded equilibrium TP dataset. However, these methods all overlook the close correlation between TPs and technical charts. For example, for head and shoulder chart, one head point and four shoulder points, which all belong to TPs, can identify the simplest head and shoulder chart. On the contrary, when we identify that the current price sequence is particularly similar to the head and shoulder chart, then whether the future price point is a TP can be determined by the future trend of the head and shoulder chart. Therefore, chart information is crucial for determining TPs.

B. Chart Similarity

Chart information is vital for determining TPs, so we need methods that can identify charts or extract chart information. The traditional methods of identifying charts employ experience or experts to propose specific judgment rules, such as “rounding bottom”, “bull flag” and “head and shoulder” [8], [28], [31]. However, traditional methods have problems such as variable judgment rules and difficulty in quantification [45]. Therefore, researchers gradually utilize similarity methods to extract chart

information, avoiding direct quantification of charts [22]. There are usually two types for similarity methods. One is to directly use sequence similarity such as Pearson Correlation, Dynamic Time Warping (DTW) [2], [37], etc. on price sequences or technical indicator sequences [45], [46], and the other is to convert price sequences into other data structures such as images [23] and graphs [30], [52], etc. for similarity judgment. Converting the original time series price data into a graph structure can avoid generating sparse data and extract general chart information [30], [52]. An illustration of how chart similarity could address the challenge of utilizing charts is shown in Fig. 2 of supplement materials.

Among these methods, the direct use of sequence similarity and the conversion of sequences into images lack robustness. For the similarity of the converted image, due to the sensitivity of the image to small changes in pixels, two images with very small differences may be judged as completely different types, which clearly does not have robustness [40]. Li et al. [30] has demonstrated through experiments that, compared to sequence similarity, such as Pearson Correlation, and DTW [37], the robustness of chart similarity by converting sequences into graphs is better. Therefore, we use chart similarity which converts sequences into graph structures to robustly predict TPs using chart information. Surprisingly, one of the components of chart similarity, PIP, can also help us solve the sample imbalance problem mentioned earlier.

C. Multipersistence

The second type of similarity mentioned above still has a crucial problem to be solved. By converting the sequence into other data structures, better prediction performance and robustness

can be achieved, but it is difficult to ensure convergence (i.e. similar charts must be similar after conversion) [50]. We hope to fundamentally solve the problem of convergence through more powerful mathematical tools.

Although multipersistence has shown promise in enhancing accuracy, processing simplicity, and convergence, its application in the field of machine learning remains largely unexplored [5], [6], [7], [35]. Multipersistence kernel [14], multipersistence landscapes [48], multipersistence images [6] based on the concept of slicing, and limiting multipersistence modules to affine rays are some notable efforts in the direction of developing multipersistence abstracts suitable for integration with machine learning models. Many important hidden temporal circumstances that cannot be retrieved using conventional techniques can be captured while ensuring convergence by the newly developed multipersistence mechanism [11], which implies that in the transformation graph, the survival time of the topology corresponding to small change will be very short or change quickly, while the survival time of the topology corresponding to TPs will be longer. Therefore, this mechanism sparks up to the spatio-temporal relationships related to TPs hidden in transformed graph sequences while ensuring convergence.

III. METHOD

In this part, we propose MPCNet, which is all specifically covered in Fig. 2. The first part is chart similarity, which is used to extract general chart information and mainly consists of two algorithms: PIP and VG. The second part is multipersistence graph neural network, which is utilized to ensure the convergence of our method and improve the accuracy of TP prediction. It mainly consists of three modules: the spatial graph convolutional layer, the feature transformation (FT) module, and the MEPSRL module. The third part is the prediction module, which is applied to capture spatio-temporal correlations among time series and obtain the hidden state of nodes at a future timestamp, mainly composed of self-attention and prediction layers. The source code for the method can be accessed under <https://github.com/lsz19960814/MPCNet>.

A. Chart Similarity

The chart similarity we utilize mainly consists of two parts, namely PIP and VG. Among them, we have made several improvements to the PIP algorithm to ensure the robustness of chart similarity while also dealing with sample imbalance issues at TPs.

1) *Our Developed PIP Method*: PIP algorithm is often utilized to identify crucial points in price series [21], [30], [46]. We take the close price as an example for convenience. Formally, let $\{cp_1, cp_2, \dots, cp_n\}$ be the close price series C_n of length n where $cp_i = (t_i, c_i)$, $\{c_1, c_2, \dots, c_n\}$ be the corresponding close prices, and $\{t_1, t_2, \dots, t_n\}$ be the corresponding time stamps. We can obtain two draft PIPs $cp_1 = (t_1, c_1)$ and $cp_n = (t_n, c_n)$. The euclidean distance d_E of each intermediate point $cp_k = (t_k, c_k)$, $t_k \in \{t_2, \dots, t_{n-1}\}$ to the two draft PIPs is defined as

$$d_E(cp_k, cp_1, cp_n) = \sqrt{(t_k - t_1)^2 + (c_k - c_1)^2} +$$

Algorithm 1: Calculating PIPs.

Input: close price series $cp_k^t = (cp_k^i, \dots, cp_k^{i+T})$, PIPs number m
Output: PIPs, chart importance score series
 $IS_i = (s_i, \dots, s_{i+T})$
1: **for** $p = i \rightarrow i + T$ **do**
2: $s_p = 0$
3: **end for**
4: add cp_k^i, cp_k^{i+T} to PIPs
5: $m^* =$ the number of PIPs; denote PIPs timestamps as $p_1, \dots, p_{m^*}$
6: **while** $m^* < m$ **do**
7: **for** $p = i \rightarrow i + T$ **do**
8: initial maximum distance $d_{max} = 0$
9: **for** $j = 2 \rightarrow m^*$ **do**
10: $c = j - 1$
11: **if** $p > p_c$ and $p < p_j$ **then**
12: p point distance $d_p = d_E(cp_k^c, cp_k^i, cp_k^{i+T})$
13: **if** $d_p > d_{max}$ **then**
14: PIPs timestamp $p^* = p$, $d_{max} = d_p$
15: **end if**
16: **end if**
17: **end for**
18: **end for**
19: add x_{p^*} to PIPs, p^* to PIPs timestamps
20: $s_{p^*} = d_{max}$
21: **end while**
22: $s_i, s_{i+T} = \max(IS_i) + 1$
23: **return** PIPs, IS_i

$$\sqrt{(t_n - t_k)^2 + (c_n - c_k)^2}. \quad (1)$$

The new PIP, $cp_k^* = (t_k^*, c_k^*)$, maximizes the distance d_E at t_k^* ,

$$k^* = \arg \max_k (d_E(cp_k, cp_1, cp_n)), \quad (2)$$

where “arg max” denotes the maximum function argument. This selection process occurs within the price sequence of a single stock and does not involve comparison across different stocks. If we set the number of PIPs to $m < n$, we can extract a key point sequence $\{cp_1, cp_{k^*}, \dots, cp_n\}$ of length m from C_n . Due to the fragile recognition rules and scales of charts [30], a price series often contains more than one chart. The representativeness of points increases as the number of required PIPs decreases. To quantify the importance of PIPs in identifying charts within a price series, we introduce the concept of the chart importance score, which is based on the maximum distance between adjacent points when they are added to PIPs. Notably, at the initial stage, the start point and stop point are preserved, resulting in the highest importance score assigned to these points. We will employ the score to filter the topology of charts in the subsequent multipersistence section. For a detailed description of the pseudocode used to calculate PIPs and the chart importance score, refer to Algorithm 1. The first to third lines of Algorithm 1 are the initialization of chart importance scores. The fourth line adds

the initial endpoint to PIP. Loop through the fifth to 21st line determines the point with the maximum distance d_E within the interval, and adds the point to the PIP, while using the maximum distance as its chart importance score. The sixth line assigns the max chart importance score to the initial endpoint.

2) *Graphs Creation*: After using the PIP algorithm to obtain a PIP sequence, we utilize the VG algorithm to convert the PIP sequence into a graph structure. Previous studies have shown that the VG of a time series remains consistent across various transformations and has the ability to differentiate between distinct graphs [27]. In essence, the VG algorithm is capable of recognizing different types of charts and exhibits robustness against diverse variations [30].

To construct the transformed graph, we consider all the data points within the key point sequence as vertices. The connectivity between these vertices follows a set of rules: we construct a vertical bar chart for the modified time series based on the associated prices. If it is possible to draw a straight line connecting any two data points without intersecting any intermediate price levels, an edge is added between the corresponding vertices in the graph. Formally, the visibility criterion [49] defines how a VG is derived from a time series. Given two arbitrary data points, denoted as (t_a^*, c_a^*) and (t_b^*, c_b^*) , where $t_a^*, t_b^* > 0$ within the time series, these points possess visibility and are represented as vertices in the corresponding graph if there exists another data point (t_c^*, c_c^*) such that $t_a^* < t_c^* < t_b^*$:

$$c_c^* < c_a^* + (c_b^* - c_a^*) \frac{t_c^* - t_a^*}{t_b^* - t_a^*}. \quad (3)$$

The robustness of combining the PIP algorithm and the VG algorithm in utilizing chart information has been demonstrated and proven in [26], [45]. PIP can extract key features for identifying charts and reduce interference caused by different variations [45]. VG adopts a set of geometric and sorting standards for scalar, real value time series, providing a comprehensive description of the underlying dynamic system. Previous studies have shown that VG can maintain its integrity even after undergoing various variations and exhibit the ability to distinguish between different types of topological fractals [26]. Moreover, we verify the robustness of chart similarity through experiments in Section VI.

3) *Balancing TP and OP Classes*: One of the critical steps in TPs prediction is labeling the TPs and ensuring class balance, which includes sample balance across different time periods and sample balance across the entire dataset, because the performance of machine learning methods is significantly hindered by imbalanced data [10], [23], [41]. Unbalanced samples can also have a negative impact on actual transactions. In previous studies [4], [17], [25], [38], a buying day was defined as the day with the lowest close price within a regular period, while a selling point was determined as the peak within the same period. The remaining days were classified as holding days, and this labeling process was repeated iteratively until all days were labeled. However, a limitation of their approach was the creation of imbalanced classes with a significantly larger number of holding days compared to buying and selling points, which will result in significant risk exposure. We refer to the methods of marking and

Algorithm 2: Balancing TP and OP Classes.

Input: close price series C_N , sub-series length n , neighbor size NS , α_{PIP} , β_{PIP}

Output: TPs

1: split C_N into sn sub-series $\{C_n^1, C_n^2, \dots, C_n^{sn}\}$

2: **for** $p = 1 \rightarrow sn$ **do**

3: $UN_{PIP}^{Max} = 0$

4: **for** $q = 2 \rightarrow n$ **do**

5: calculate $PIPs_q$ of C_n^p with PIP number q

6: calculate UN_{PIP}^q of C_n^p with $PIPs_q$, α_{PIP} and β_{PIP}

7: **if** $UN_{PIP}^q > UN_{PIP}^{Max}$ **then**

8: $q_{max} = q$

9: $UN_{PIP}^{Max} = UN_{PIP}^q$

10: **end if**

11: **end for**

12: **end for**

13: add $PIPs_{q_{max}}$ to TPs

14: **if** $NS = 1$ **then**

15: add history neighbors of TPs to TPs

16: **else**

17: add history neighbors of TPs and history neighbors of TPs' history neighbors to TPs

18: **end if**

19: **return** TPs

balancing TPs in the [10], [23], [41]. In order to achieve overall sample balance, on the one hand, we use fitness functions in our developed PIP algorithm to balance the proportion of TPs in different time periods, and on the other hand, we increase the number of TPs in all time periods by adding neighbors as the TPs.

First, we balance the proportion of TPs in different time periods, as the fluctuation levels of price series vary within different time periods. Time periods with higher volatility contain more TPs, while time periods with lower volatility contain fewer TPs. The selection of an appropriate number significantly impacts the performance of PIPs, thereby influencing the balance of TPs and OPs. A lower number corresponds to a reduced number of generated TPs, whereas a higher number results in a greater abundance of TPs [23], [41]. In this study, we introduce a fitness function, denoted as UN_{PIP} , which acquires the adjusted number of TPs derived from the peaks and troughs identified by PIP algorithm. The optimization of this fitness function, as illustrated in (4), allows us to determine the optimal threshold value:

$$UN_{PIP} = m - \alpha_{PIP} \sum_{i=2}^m \text{sign}(\max(\beta_{PIP} - (t_i - t_{i-1}), 0)), \quad (4)$$

where α_{PIP} (set to 0.1 in this study) serves as a constant penalty factor. Additionally, β_{PIP} (assigned a value of 15 in this study) denotes the maximum duration within which trades are deemed favorable, and $t_i - t_{i-1}$ refers to the time span between two consecutive PIPs. Here, m represents the total number of PIPs. The objective is to iterate through different PIP number m ,

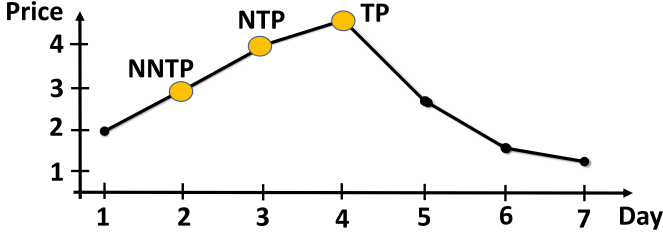


Fig. 3. The example of balance TPs.

calculate PIPs as like in Algorithm 1, and identify the optimal PIP number m that maximizes the value of UN_{PIP} . We utilize the calculated PIPs as TPs as like [10], [23], [41].

In accordance with the presented fitness function, the number of TPs (PIPs) decreases as the fluctuation of the stock price series increases. Conversely, a higher level of fluctuation in the stock price series leads to an decrease in the number of TPs. Therefore, the utilization of appropriate balancing algorithms balances the proportion of TPs in price sequences of different periods with various levels of volatility. One of the main advantages of PIP is its flexibility, unlike the approach used in [4], [17], [25], [38], as it allows for both short-term and long-term trades based on the fluctuations of the close price. In other words, there is no requirement to have a trade within a strict regular time window, which was the underlying assumption in the previous studies [4], [17], [25], [38].

Then we balance the sample proportion of the entire dataset. In this study, we introduce a novel balancing algorithm that incorporates an oversampling technique [23], [41]. Unlike previous approaches, we specifically focus on considering the history neighbors of a TP as additional TPs, resulting in an increased number of TPs. For example, in Fig. 3, if we assume a neighbor size of one ($NS = 1$) and designate the fourth day as a TP, then the preceding third day are transformed into additional TPs (NTPs). The reason why we only expand on historical neighbors here is from a practical perspective, because if investors can accurately predict TP's historical neighbors, then in actual transactions, investors can formulate strategies in advance for operation. However, predicting TP's future neighbors is meaningless for actual transactions, as the TP has already passed. Overall, the steps for balancing TPs are shown in the Algorithm 2. The first to 13th lines of the Algorithm 2 are to iterate through different PIP number m , calculate PIPs as like in Algorithm 1, and identify the optimal PIP number m that maximizes the value of UN_{PIP} , which can balance the proportion of TPs in different time periods. The 14th to 18th lines of the Algorithm 2 are to balance the sample proportion of the entire dataset.

B. Multipersistence Euler-Poincaré Surfaces

By applying the aforementioned chart similarity, we can convert the close price series C_N^t at time point t , which has a length of N , into a graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, A_t, \mathbf{X}_t)$. Here, $\mathcal{V}_t$ represents a set of nodes, and $\mathcal{E}_t$ represents a set of edges. The sizes of $\mathcal{V}_t$ and $\mathcal{E}_t$ are denoted as $|\mathcal{V}_t| = m$ and $|\mathcal{E}_t| = M_t$, respectively. m and M_t represent the number of nodes and

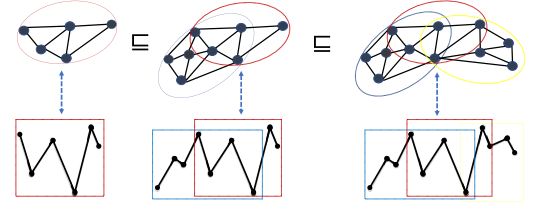


Fig. 4. An illustration of how PH can find key chart information. We have circled three different charts with rectangular boxes of different colors. Yellow rectangular: Bull flag; Red rectangular: Rounding bottom; Blue rectangular: Head and shoulder.

edges in the transformed graph $\mathcal{G}_t$ at timestamp t . The adjacency matrix $A_t \in \mathbb{R}^{m \times m}$ and the node feature matrix $\mathbf{X}_t = \{x_{t,1}, x_{t,2}, \dots, x_{t,m}\}^T \in \mathbb{R}^{m \times F_N}$ have a feature dimension of F_N . We consider a total of T time steps. Using a sequence of observations $\mathcal{X} = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_T\} \in \mathbb{R}^{T \times m \times F_N}$ with T timestamps and F_N node characteristics, we construct spatio-temporal networks $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$.

1) *Single Persistence*: The complex data's shape can be determined along a single geometric dimension using persistent homology (PH), which is based on a single parameter. In order to examine a graph $\mathcal{G}_t$ as a series of nested sub-graphs, or a graph filtration, $\mathcal{G}_t^1 \subseteq \mathcal{G}_t^2 \dots \subseteq \mathcal{G}_t^{s_m} = \mathcal{G}_t$, caused by this changing scale parameter, one must first choose a relevant parameter of interest. All extracted topological features can be then summarized as a multiset in $\mathbb{R}^2$, called persistence diagram (PD): $\mathcal{D}_k(\mathbf{C}) = \{(p_i, q_i) \in \mathbb{R}^2 \mid p_i < q_i\}$, where birth p_i and death q_i mark the filtration indexes at which the k -dimensional topological feature ρ_i appears and disappears, respectively. The farther (p_i, q_i) from the diagonal is (that is, the more persistent ρ_i is), the likelier ρ_i is to contain salient information on $\mathcal{G}_t$. Multiplicity of $(x_p, y_q) \in \mathcal{D}$ is the number of k -dimensional topological features (k -holes) that are born and die at x_p and y_q , respectively. Features on the diagonal of $\mathcal{D}$ have infinite multiplicity. As shown in the Fig. 4, we demonstrate how PH can find key chart information. The far right side of the Fig. 4 shows the original price sequence and its transformed graph structure. As we can see, we have circled three different charts with rectangular boxes of different colors. From right to left is the process of fetching subgraphs and the corresponding price subsequence of subgraphs, as PH does. Finally, on the far left side, we obtained the most critical chart information, including the rounding bottom chart and its corresponding subgraph structure.

Utilizing this filtration process, we gain the ability to identify the emergence and disappearance of various structural patterns, including loops and cavities. Moreover, we are able to monitor the duration of their existence. We construct a simplicial complex $\mathbf{P}_t^i$ from each subgraph $\mathcal{G}_t^i$ to provide a filtration $\mathbf{P}_t^1 \subseteq \mathbf{P}_t^2 \dots \subseteq \mathbf{P}_t^{s_m}$ (e.g., clique complexes) to increase the efficiency and convergence of the counting process. For example, to construct such a filtration, one common approach is to employ a filtering function $f: \mathcal{V}_t \mapsto \mathbb{R}$ along with a series of ascending thresholds $\{\gamma_i\}_{i=1}^{s_m}$. This allows us to define the rising sets $\mathbf{P}_t^i = \{\Delta \in \mathbf{P}_t : \max_{v \in \Delta} f(v) \leq \gamma_i\}$. A sublevel set filtration of f is the resulting construction, and f may be chosen as a degree, centrality, or eccentricity function.

2) *Multipersistence*: Data in many applications, especially those involving spatio-temporal modeling, may be naturally indexed by multiple parameters, such as real-time transaction volume and transaction network nature in cryptocurrency analysis. Alternatively, the main focus may be on identifying the shape characteristics of complex data along multiple dimensions. For example, in order to better predict stock price trends and mitigate risks, we may need to evaluate the structural patterns of asset portfolio dynamics, not along one-dimensional dimensions, but along both technical indicators and charts, as a measure of key investors' shared perception of stock market returns and volatility. This multidimensional analysis in terms of topology and geometric properties can be solved by extending the PH based on a single filter to the case of multiple filters. That is to say, the idea of multipersistence is based on the multivariate filtering function $F: \mathcal{V}_t \mapsto \mathbb{R}^d$ to simultaneously evaluate the shape features of $\mathcal{G}_t$. As a result, for example, for $d = 2$ and a set of nondecreasing thresholds $\{\gamma_i\}_1^{s_m}$ and $\{\eta_j\}_1^{s_n}$, we obtain a bifiltration of complexes $\{\mathbf{P}_t^{\gamma_i, \eta_j} \mid 1 \leq i \leq s_m, 1 \leq j \leq s_n\}$ such that if $\eta_k < \eta_l$, then $\mathbf{P}_t^{\gamma_i, \eta_k} \hookrightarrow \mathbf{P}_t^{\gamma_i, \eta_l}$ and if $\gamma_i < \gamma_j$, then $\mathbf{P}_t^{\gamma_i, \eta_k} \hookrightarrow \mathbf{P}_t^{\gamma_j, \eta_k}$. The bigraded multipersistence module $\{H_k(\mathbf{P}_t^{\gamma_i, \eta_j})\}$, where H_k is the k^{th} homology group, is the result of this bifiltration. We suggest a new multipersistence summary, called a Multipersistence Euler-Poincaré Surface, which is motivated by [3], [11].

Definition 1: Multipersistence Euler-Poincaré Surface. Let $\{\mathcal{G}_t\}_{t=1}^T$ be a collection of graphs that change over time. Let $\mathcal{I} = \{(\gamma_i, \eta_j) \mid 1 \leq i \leq s_m, 1 \leq j \leq s_n\}$ be the thresholds for the multivariate filtering function $F: \mathcal{V}_t \mapsto \mathbb{R}^2$. Let $\chi(\mathbf{P})$ be the Euler-Poincaré characteristic, which is defined as the alternating sum of the number of k -simplices of a given simplicial complex $\mathbf{P}$, like $\chi(\mathbf{P}) = \sum_{k=0}^{s_M} (-1)^k \text{Bet}_k(\mathbf{P})$, where s_M is $\mathbf{P}$'s largest homological dimension and $\text{Bet}_k(\mathbf{P})$ is $\mathbf{P}$'s k^{th} Betti number (the first Betti number represents the number of connected components in topology). Let $\mathbf{P}_t^{\gamma_i, \eta_j}$ be the clique complex of the induced subgraph $\mathcal{G}_t^{\gamma_i, \eta_j} = F^{-1}((-\infty, \gamma_i] \times (-\infty, \eta_j])$, $t = 1, 2, \dots, T$. The Multipersistence Euler-Poincaré Surface (MEPS) is a collection of time-evolving $s_m \times s_n$ -matrices $\{\mathbb{EP}_t^{\gamma_i, \eta_j}\}_{t=1}^T$ such that $\mathbb{EP}_t^{\gamma_i, \eta_j} = \chi(\mathbf{P}_t^{\gamma_i, \eta_j})$ for $1 \leq i \leq s_m, 1 \leq j \leq s_n$. (A toy illustration of how MEPS is calculated is shown in Fig. 1).

3) *Theoretical Guarantees of MEPS*: Take into account two graphs, $\mathcal{G}^+$ and $\mathcal{G}^-$, where the time index t is suppressed for conciseness. Let $\mathbf{P}^\pm$ represent the clique complexes of $\mathcal{G}^\pm$, and let $\hat{\mathbf{P}}^\pm = \{\mathbf{P}_{ij}^\pm\}$ represent the bifiltration brought about by $(\mathbf{P}^\pm, F, \mathcal{I})$ as before. Let Euler-Poincaré Surfaces, or the equivalent $s_m \times s_n$ matrices, be $\mathbb{EP}^\pm$. The distance between $\mathbb{EP}^+$ and $\mathbb{EP}^-$ is then set to $\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1} = \sum_{i=1}^{s_m} \sum_{j=1}^{s_n} |\mathbb{EP}_{ij}^+ - \mathbb{EP}_{ij}^-|$, where $\|\cdot\|_{1,1}$ is the vectorized L_1 matrix norm.

Due to the nature of the $\mathbb{EP}^\pm$ [43], we now present a L_1 -based multipersistence metric rather than utilizing L_∞ -based metrics like, for example, matching or interleaving. Let $\mathcal{D}_k^f(\mathbf{P}^\pm)$ and $\mathcal{D}_k^g(\mathbf{P}^\pm)$ be the k^{th} single persistence diagrams (PDs) of $\mathbf{P}^\pm$ for filtrations induced by functions $f, g: \mathcal{V}^\pm \mapsto \mathbb{R}$, respectively. Let $\mathbf{P}_{i*}^\pm$ and $\mathbf{P}_{*j}^\pm$ be clique complexes corresponding to $\mathcal{G}_{i*}^\pm = f^{-1}((-\infty, \gamma_i])$ and $\mathcal{G}_{*j}^\pm = g^{-1}((-\infty, \eta_j])$. Define the i^{th} column distance for the k^{th} PDs as $\text{Dis}_{i*}^k(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) =$

$\mathcal{W}_1(\mathcal{D}_k^g(\mathbf{P}_{i*}^+), \mathcal{D}_k^g(\mathbf{P}_{i*}^-))$, where $\mathcal{W}_1$ is the Wasserstein-1 distance [16]. Similarly, the j^{th} row distance for k^{th} PDs is $\text{Dis}_{*j}^k(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) = \mathcal{W}_1(\mathcal{D}_k^f(\mathbf{P}_{*j}^+), \mathcal{D}_k^f(\mathbf{P}_{*j}^-))$.

Definition 2: Weak L_1 -metric for multipersistence. The weak L_1 -metric between $\hat{\mathbf{P}}^\pm$ is

$$\text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) = \max\{\text{Dis}_c(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-), \text{Dis}_r(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-)\} \quad (5)$$

$$\text{s.t. } \text{Dis}_c(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) = \sum_{k=0}^{s_M} \sum_{i=1}^{s_m} \text{Dis}_{i*}^k(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) \text{ and } \text{Dis}_r(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) = \sum_{k=0}^{s_M} \sum_{j=1}^{s_n} \text{Dis}_{*j}^k(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-).$$

We are now ready to present our convergence finding for Euler-Poincaré Surfaces:

Theorem 1: Considering the given definitions of $\mathcal{G}^\pm$, F , $\hat{\mathbf{P}}^\pm$, and $\mathbb{EP}^\pm$, it can be concluded that the Euler-Poincaré Surfaces exhibit convergence with respect to the weak L_1 -metric. In other words, $\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1}$ is bounded by $C \cdot \text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-)$, where C is a positive constant.

Proof: We begin by presenting the proof's outline. You should be aware that Euler-Poincaré Surfaces are the alternating sum of the Betti numbers $\{\text{Bet}_k\}$. Therefore, the difference between the respective Betti numbers for $\hat{\mathbf{P}}^\pm$, i.e., (6) and (7), is what determines the distance $\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1}$. Our major claim is based on fixing i and k and taking column distance Dis_c into account:

$$\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1} \leq C \cdot \text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-), \quad C > 0,$$

then simplifies to (9) and (10). To explain the disparity, $|\text{Bet}_k(\mathbf{P}_j^+) - \text{Bet}_k(\mathbf{P}_j^-)|$ to $\mathcal{W}_1(\mathcal{D}(\mathbf{P}^+), \mathcal{D}(\mathbf{P}^-))$, we note that the Betti number $\text{Bet}_k(\mathbf{P}_j^\pm)$ is equal to the total number of barcodes in $\mathcal{D}(\mathbf{P}^\pm)$, which contains η_j , i.e., $\#\{r \mid \eta_j \in [p_r^\pm, q_r^\pm)\}$ ((10) and (11)). However, the Wasserstein-1 distances of $\mathcal{D}(\mathbf{P}^+)$ and $\mathcal{D}(\mathbf{P}^-)$, i.e. (12) and (13), can be used to limit variances in the number of barcodes containing η_j for $\mathbf{P}^+$ and $\mathbf{P}^-$. The proof is completed by adding these phrases together on the indices i and k .

We will now discuss derivations in more detail. Given $\mathbb{EP}_{ij}^\pm = \chi(\mathbf{P}_{ij}^\pm)$, we have $\mathbb{EP}_{ij}^\pm = \sum_{k=0}^{s_M} \text{Bet}_k(\mathbf{P}_{ij}^\pm)$. As a result, for any i, j ,

$$\begin{aligned} |\mathbb{EP}_{ij}^+ - \mathbb{EP}_{ij}^-| &= \left| \sum_{k=0}^{s_M} (-1)^k [\text{Bet}_k(\mathbf{P}_{ij}^+) - \text{Bet}_k(\mathbf{P}_{ij}^-)] \right| \\ &\leq \sum_{k=0}^{s_M} |\text{Bet}_k(\mathbf{P}_{ij}^+) - \text{Bet}_k(\mathbf{P}_{ij}^-)|. \end{aligned} \quad (6)$$

Since $\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1} = \sum_{i=1}^{s_m} \sum_{j=1}^{s_n} |\mathbb{EP}_{ij}^+ - \mathbb{EP}_{ij}^-|$, we have

$$\|\mathbb{EP}^+ - \mathbb{EP}^-\|_{1,1} \leq \sum_{i=1}^{s_m} \sum_{j=1}^{s_n} \sum_{k=0}^{s_M} |\text{Bet}_k(\mathbf{P}_{ij}^+) - \text{Bet}_k(\mathbf{P}_{ij}^-)|. \quad (7)$$

Now, we must bind the later sum by the multipersistence distance $\text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-)$ in order to finish the proof. Remember that Definition 2 states:

$$\begin{aligned} \text{Dis}_c(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-) &= \sum_{k=0}^{s_M} \sum_{i=1}^{s_m} \mathcal{W}_1(\mathcal{D}_k^g(\mathbf{P}_{i*}^+), \mathcal{D}_k^g(\mathbf{P}_{i*}^-)) \\ &\leq \text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-). \end{aligned} \quad (8)$$

Then, in order to demonstrate the theorem using (7), it is sufficient to demonstrate that for any $0 \leq k_0 \leq s_M$ and for any $1 \leq i_0 \leq s_m$:

$$\sum_{j=1}^{s_n} |\text{Bet}_{k_0}(\mathbf{P}_{i_0j}^+) - \text{Bet}_{k_0}(\mathbf{P}_{i_0j}^-)| \leq C \cdot \mathcal{W}_1(\mathcal{D}_{k_0}^g(\mathbf{P}_{i_0*}^+), \mathcal{D}_{k_0}^g(\mathbf{P}_{i_0*}^-)), \quad C > 0. \quad (9)$$

For ease of use without sacrificing generality, we fix $1 \leq i_0 \leq s_m$ and $0 \leq k_0 \leq s_M$. Additionally, we omit the subscripts and superscripts, i.e. $\mathbf{P}^\pm = \mathbf{P}_{i_0*}^\pm$ and $\mathbf{P}_j^\pm = \mathbf{P}_{i_0j}^\pm$, $\mathcal{D}(\mathbf{P}^\pm) = \mathcal{D}_{k_0}^g(\mathbf{P}_{i_0*}^\pm)$ and $\text{Bet}() = \text{Bet}_{k_0}()$. For the sake of notations, (9) is equivalent to in these condensed notations:

$$\sum_{j=1}^{s_n} |\text{Bet}(\mathbf{P}_j^+) - \text{Bet}(\mathbf{P}_j^-)| \leq C \cdot \mathcal{W}_1(\mathcal{D}(\mathbf{P}^+), \mathcal{D}(\mathbf{P}^-)) \quad C > 0. \quad (10)$$

The relationship between the Betti numbers $\text{Bet}(\mathbf{P}_j^\pm)$ and the persistence diagrams $\mathcal{D}(\mathbf{P}^\pm)$ must be assessed in order to demonstrate (10). Think about $\mathcal{D}(\mathbf{P}) = \{(p_r, q_r) \mid 1 \leq r \leq Q\}$, where p_r indicates the time at which a k_0 -cycle ρ_r in $\mathbf{P}$ is born, and q_r represents the time at which ρ_r dies. ρ_r specifically produces a barcode $[p_r, q_r)$. Define the indicator function $\psi_r(t) = \mathcal{I}_{[p_r, q_r)}(t)$ for each of these ρ_r i.e., $\psi_r(t) = 1$ for $t \in [p_r, q_r)$ and $\psi_r(t) = 0$ otherwise. So, $\text{Bet} : [\eta_1, \infty) \rightarrow \mathbb{N}$ such that $\text{Bet}(t) = \sum_{r=1}^Q \psi_r(t)$ for $t \in [\eta_1, \infty)$ is the definition of the Betti function of $\mathbf{P}$. (In this case, among the set of thresholds $\mathcal{I} = \{(\gamma_i, \eta_j) \mid 1 \leq i \leq s_m, 1 \leq j \leq s_n\}$, the threshold η_1 corresponds to the minimum value among all thresholds associated with the filtering function g . Similar to this, if we apply the other filtering function f and take into account row sum Dis_r rather than column sum Dis_c , $\text{Bet} : [\gamma_1, \infty) \rightarrow \mathbb{N}$.) Hence,

$$\text{Bet}(\mathbf{P}_j^\pm) = \sum_{r=1}^{Q^\pm} \psi_r^\pm(\eta_j), \quad j \in \mathbb{Z}^+.$$

Consequently, we obtain

$$\begin{aligned} \sum_{j=1}^{s_n} |\text{Bet}(\mathbf{P}_j^+) - \text{Bet}(\mathbf{P}_j^-)| &= \sum_{j=1}^{s_n} \left| \sum_{r=1}^{Q^+} \psi_r^+(\eta_j) - \sum_{r=1}^{Q^-} \psi_r^-(\eta_j) \right| \\ &\leq \sum_{j=1}^{s_n} \sum_{r=1}^{\hat{Q}} |\psi_r^+(\eta_j) - \psi_{\phi(r)}^-(\eta_j)|, \end{aligned} \quad (11)$$

where $\phi : \mathcal{D}(\mathbf{P}^+) \rightarrow \mathcal{D}(\mathbf{P}^-)$ represents the optimal matching within the framework of $\mathcal{W}_1(\mathcal{D}(\mathbf{P}^+), \mathcal{D}(\mathbf{P}^-))$, and $\hat{Q}$ is defined as the maximum of Q^+ and Q^- .

Please note that the count ($\#r$) of intervals $[p_r^+, q_r^+) \triangle [p_{\phi(r)}^+, p_{\phi(r)}^-)$ that include the interval $[\eta_j, \eta_{j+1})$, where $\triangle$ represents the symmetric difference of sets, is fewer than the contribution of each $|\psi_r^+(\eta_j) - \psi_{\phi(r)}^-(\eta_j)|$ to the total at η_j . Let $K = \min_j(\eta_{j+1} - \eta_j)$. To guarantee that each of these intervals contributes at least one to the Betti number count, we divide the length $\|[p_r^+, q_r^+) \triangle [p_{\phi(r)}^+, p_{\phi(r)}^-)\|$ by K . This ensures that each

interval makes a minimum unitary contribution. Consequently, we reach the following significant inequality:

$$\begin{aligned} \sum_{j=1}^{s_n} \sum_{r=1}^{\hat{Q}} |\psi_r^+(\eta_j) - \psi_{\phi(r)}^-(\eta_j)| \\ \leq \frac{1}{K} \cdot \sum_r |p_r^+ - p_{\phi(r)}^-| + |q_r^+ - q_{\phi(r)}^-|. \end{aligned} \quad (12)$$

Finally, since

$$\sum_{r=1}^{\hat{Q}} |p_r^+ - p_{\phi(r)}^-| + |q_r^+ - q_{\phi(r)}^-| \leq 2\mathcal{W}_1(\mathcal{D}(\mathbf{P}^+), \mathcal{D}(\mathbf{P}^-)),$$

we obtain

$$\sum_{j=1}^{s_n} |\text{Bet}^+(\eta_j) - \text{Bet}^-(\eta_j)| \leq \frac{2}{K} \cdot \mathcal{W}_1(\mathcal{D}(\mathbf{P}^+), \mathcal{D}(\mathbf{P}^-)). \quad (13)$$

Since (13) holds for any $1 \leq i_0 \leq s_m$ and $0 \leq k_0 \leq s_M$, for $C = 2/K$ we get

$$\sum_{i=1}^{s_m} \sum_{j=1}^{s_n} \sum_{k=0}^{s_M} |\text{Bet}_k(\mathbf{P}_{ij}^+) - \text{Bet}_k(\mathbf{P}_{ij}^-)| \leq C \cdot \text{Dis}_c(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-).$$

By (7), this suggests

$$\|\mathbb{E}\mathbf{P}^+ - \mathbb{E}\mathbf{P}^-\|_{1,1} \leq C \cdot \text{Dis}(\hat{\mathbf{P}}^+, \hat{\mathbf{P}}^-),$$

which completes the proof. $\square$

This convergence conclusion suggests that the distances between the resulting Euler-Poincaré Surfaces are determined by the distance between multiparameter PDs. Frequently used notations in the above proof are summarized in Section II of supplementary material. One can draw the conclusion that modest changes in the multipersistence filtering function $F : \mathcal{V}_t \mapsto \mathbb{R}^d$ or small changes in the input data can only cause a small change in MEPS surfaces by combining this with the convergence result for PDs [11], [13].

C. Multipersistence Graph Convolutional Network

By considering $\mathcal{T}$ previous observations $\mathcal{X}^\mathcal{T} = \{\mathbf{X}_{t-\mathcal{T}}, \mathbf{X}_{t-\mathcal{T}+1}, \dots, \mathbf{X}_{t-1}\} \in \mathbb{R}^{\mathcal{T} \times m \times F_N}$, our objective is to train a multivariate forecasting model capable of predicting future TP observations at the subsequent time step.

1) *Graph Learning Architecture:* Our MPCNet's graph representation learning is based on GCN. GCN-based techniques utilize the input comprising the adjacency matrix A_t and the node feature matrix X_t of the original input graph $\mathcal{G}_t$. This allows for the learning of node representations by considering both the graph topology and node features. The prior understanding of the network structure, however, (1) could limit the modeling ability (i.e., graph edges cannot encode all the complicated chart graph) and (2) causes the disregard of nearby information with high diversity. To address these limitations, we overcome them by training the self-adaptive adjacency matrix $\mathbf{S} = \{s_{uv}\}_{m \times m}$ normalized and based on the pre-defined "cost" of remaining in the same node. This is achieved through the utilization of the learnable node embedding $\mathbf{E}_\phi = (e_{1,\phi}, e_{2,\phi}, \dots, e_{m,\phi})^\top \in \mathbb{R}^{m \times d_c}$,

drawing inspiration from the concept of adaptive dependency matrices [9]:

$$s_{uv} = \begin{cases} \frac{\exp(\text{ReLU}(e_{u,\phi} e_{v,\phi}^\top))}{\sum_{v \in \mathcal{V} \setminus \{u\}} \exp(\text{ReLU}(e_{u,\phi} e_{v,\phi}^\top)) + \exp(d_{uu})}, & u \neq v, \\ \frac{d_{uu}}{\sum_{v \in \mathcal{V} \setminus \{u\}} \exp(\text{ReLU}(e_{u,\phi} e_{v,\phi}^\top)) + \exp(d_{uu})}, & u = v. \end{cases} \quad (14)$$

Here, the activation function $\text{ReLU}(\cdot) = \max(0, \cdot)$ ensures $s_{uv} \geq 0$ and the hyperparameter d_{uu} signifies the “cost” associated with staying in the current node u . Utilizing S as the normalized Laplacian is viable due to the properties that $\sum_v s_{uv} = 1$ and $s_{uv} \geq 0$. We express graph diffusion as a matrix power series to enable more efficient and reliable learning of both spatial and spectral features. Specifically, let $\tilde{S}$ be an $m \times m \times K$ Laplacian tensor that comprises the power series $\{\mathbf{I}, \mathbf{S}, \dots, \mathbf{S}^{K-1}\}$ of S , where $K \geq 2$. Here, $\mathbf{I} \in \mathbb{R}^{m \times m}$ denotes the identity matrix.

2) *Spatio-Temporal Feature Transformation*: Be aware that information types in the spatial and temporal domains are highly varied but interrelated. By transforming the initial input feature space into a high-level latent feature space, we can effectively extract information from multiple informative sources. This approach draws inspiration from statistical factor analysis and can be described as follows:

$$\widehat{\mathbf{H}}_{i,FT}^{(\ell+1)} = (\mathcal{X}^\top \mathbf{E}_\phi \Theta_{FT}^{(\ell)})^\top \mathbf{U}, \quad (15)$$

where $\Theta_{FT}^{(\ell)} \in \mathbb{R}^{d_c \times F_N \times Q_{FT}}$ and $\mathbf{U} \in \mathbb{R}^T$ represent the learnable parameters. Finally, we reshape $\widehat{\mathbf{H}}_{i,FT}^{(\ell+1)} \in \mathbb{R}^{Q_{FT} \times m}$ to $\mathbf{H}_{i,FT}^{(\ell+1)} \in \mathbb{R}^{m \times Q_{FT}}$.

3) *Spatial Graph Convolutional Layer*: We now shift to building the spatial graph convolution via the multi-hops neighborhood of each node to aggregate the features of each node and yield node embeddings

$$\mathbf{H}_{i,Spa}^{(\ell+1)} = \widehat{\mathbf{H}}_{i,Spa}^{(\ell)} \mathbf{E}_\phi \Theta_{Spa}^{(\ell)}, \quad (16)$$

where we reshape $(\tilde{S}^\top \mathbf{H}_{i,Spa}^{(\ell)})^\top \in \mathbb{R}^{K \times m \times F_N}$ to $\widehat{\mathbf{H}}_{i,Spa}^{(\ell)} \in \mathbb{R}^{m \times K \times F_N}$, $\Theta_{Spa}^{(\ell)} \in \mathbb{R}^{d_c \times K \times F_N \times Q_{Spa}}$ is the trainable weight, $\tilde{S}$ is the Laplacian tensor, $\mathbf{H}_{i,Spa}^{(\ell)} \in \mathbb{R}^{m \times F_N}$ and $\mathbf{H}_{i,Spa}^{(\ell+1)} \in \mathbb{R}^{m \times Q_{Spa}}$ are the node representations at the ℓ -th layer and $(\ell + 1)$ -th layer, respectively, and $\mathbf{H}_{i,Spa}^{(0)} = \mathbf{X}_i$, i.e., features at time step i of the node. We substitute weight sharing matrix factorization for matrix entry assignments to cut down on computational costs. As a result, we use a back propagation approach to promptly update the variables’ most recent state and lower the likelihood of overfitting.

4) *Learning Multipersistence Representation of Topological Features*: To fully utilize the chart information obtained from the multipersistence representation method, we propose the Multipersistence Euler-Poincaré Surface Representation Learning (MEPSRL) module, which draws inspiration from CNN architecture. The MEPSRL module combines a CNN base model f_θ , which includes convolutional and pooling layers, with a readout layer. This integration allows for comprehensive utilization of the chart information during the representation learning process. The proposed MEPSRL module delivers the following distinctive advances, in sum: the proposed approach

involves three key steps: (i) capturing global information to extract features, (ii) learning the relationship between MEPS sequences before and after a specific point by treating them as multi-channels within a fixed-size sliding window, and (iii) aggregating topological features. The MEPSRL module’s consolidated output feature is provided by

$$\Phi_{i,MP}^{(\ell+1)} = \oplus(f_{\text{GAP}}(f_{\theta_1}(\{\mathbb{E}\mathbb{P}^i\}_{i=1}^T)), f_{\text{GMP}}(f_{\theta_2}(\{\mathbb{E}\mathbb{P}^i\}_{i=1}^T))), \quad (17)$$

where we utilize various functions for feature extraction: $f_{\text{GAP}}(\cdot)$, $f_{\text{GMP}}(\cdot)$, and f_{θ_j} , where $j = \{1, 2\}$, represents the j -th CNN-based model. $f_{\text{GAP}}(\cdot)$ and $f_{\text{GMP}}(\cdot)$ functions perform global average pooling, global max pooling, respectively and $f_{\text{GAP}}(\cdot)$ generates features for each channel. The concatenation operation $\oplus$ is used to combine the outputs. Notably, we observe that $f_{\text{GMP}}(\cdot)$ enhances the representation learning of the time-conditioned multipersistence features. The resulting matrix $\Phi_{i,MP}^{(\ell+1)} \in \mathbb{R}^{Q_{MP}}$ corresponds to the output of the MEPSRL module.

We integrate the above embeddings to produce the final embedding, which further learns and fuses several latent representations from various angles, such as spatial information, spatio-temporal correlations, and persistent chart properties:

$$\mathbf{Z}_{i,out}^{(\ell+1)} = \mathcal{F}(\mathbf{H}_{i,Spa}^{(\ell+1)}, \Upsilon_i^{(\ell+1)}, \mathbf{H}_{i,FT}^{(\ell+1)}). \quad (18)$$

We incorporate global chart and global spatio-temporal information by applying $\Upsilon_i^{(\ell+1)} = (1/T \sum \tilde{\mathcal{X}}^\top) \Phi_{i,MP}^{(\ell+1)}$. In this process, we reshape $\mathcal{X}^\top \in \mathbb{R}^{T \times m \times F_N}$ to $\tilde{\mathcal{X}}^\top \in \mathbb{R}^{m \times T \times F_N}$, followed by averaging $\tilde{\mathcal{X}}^\top$ over the temporal dimension. Additionally, $\mathcal{F}(\cdot, \cdot, \cdot)$ represents a dimension-wise concatenation function applied to the embedding along the output dimension, and the summation of Q_i for $i \in \{\text{Spa}, \text{Mp}, \text{FT}\}$ equals Q_{out} .

Good filtering should have a rich range to achieve fine resolution, and it should reflect an important property related to TP prediction problems. For a good combination of two such filtering functions in multipersistence, unrelated functions should be chosen. One method is to select a function from the TP prediction domain, and the other is to select from the strict graph property (such as degree, intermediate number, etc.). By using filtering procedures based on node degree (Deg), betweenness centrality (Btwns), and PIP important score (PIS), we evaluate the chart properties of $\mathcal{G}_t$. The multivariate filtering function $F: \mathcal{V}_t \mapsto \mathbb{R}^d$ with $d = 2$ is the subject of our investigation. The set of nondecreasing thresholds for Deg, PIS, and Btwns all operate on values across nodes. It is worth noting that each pair of combinations of the filtering function generates a different multipersistence representation in the form of an Euler-Poincaré surface, which affects the performance of the algorithm. We only selected the best combination result.

5) *Modeling the Spatio-Temporal Dynamics*: In order to capture spatio-temporal correlations among time series and obtain the hidden state of nodes at a future timestamp, we pass the final embedding $\mathbf{Z}_{i,out}^{(\ell+1)}$ through a self-attention layer, as depicted in the right panel of Fig. 2. Meanwhile, we further elaborate on our motivation for adding this module through examples of actual charts in Section IV of supplementary material.

As the first self-attention floor, $\mathbf{Z}_{i, \text{out}}^{(\ell+1)} \in R^{m \times Q_{\text{out}}}$ is first transformed into three feature spaces (Q, K, and V) using (19).

$$\begin{aligned} Q(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) &= \mathbf{Z}_{i, \text{out}}^{(\ell+1)} \cdot W_Q, \\ K(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) &= \mathbf{Z}_{i, \text{out}}^{(\ell+1)} \cdot W_K, \\ V(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) &= \mathbf{Z}_{i, \text{out}}^{(\ell+1)} \cdot W_V, \end{aligned} \quad (19)$$

where $W_Q, W_K \in R^{Q_{\text{out}} \times f_a}$ and $W_V \in R^{Q_{\text{out}} \times f_v}$. The channel numbers f_a for $Q(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})$ and $K(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})$ must match. The number of channels f_v for $V(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})$ may differ from f_a . Here, $Q(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) \in R^{m \times f_a}$, $K(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) \in R^{m \times f_a}$, and $V(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) \in R^{m \times f_v}$ are collectively referred to as the query space, key space, and value space, respectively. The names of the three feature spaces follow the convention [47], where attention denotes the degree to which a mapping between a prospective query and the key-value pairs in the data is recorded.

Within the second self-attention layer, the self-attention for a specific variable, denoted as $\alpha \in R^{m \times m}$, is calculated based on the features in the query space and key space using two approaches outlined in (19) and (20).

$$\mathbf{P} = Q(\mathbf{Z}_{i, \text{out}}^{(\ell+1)}) \cdot K(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})^T. \quad (20)$$

To normalize the attention, $\mathbf{P} \in R^{m \times m}$ is subjected to the softmax function, as shown in (21):

$$\alpha = \text{softmax}(\mathbf{P}). \quad (21)$$

The features in the value space $V(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})$ are subjected to the normalized attention α in the third self-attention phase. This process draws attention to hidden states $V_a = \alpha \cdot V(\mathbf{Z}_{i, \text{out}}^{(\ell+1)})$, where $V_a \in R^{m \times f_v}$.

In order to get the classification prediction, we connect the attention layer to a full connection layer. When presented with a set of B samples from the stock price series, our model utilizes cross entropy as a measure to calculate the loss between the actual label distribution y_n and the predicted distribution $\hat{y}_n$:

$$J = -\frac{1}{B} \sum_{p=1}^B y_p \log(\hat{y}_p). \quad (22)$$

IV. EXPERIMENTS

A. Data

The dataset obtained in this study includes daily volume and price data of SZ-50 and CSI-300 index constituent stocks, which are the most important part of the Chinese A-share market. We collect the daily prices of these two indexes from 2010 to 2020 and then use these basic price data to calculate nine types of transaction indicators [30], whose calculation formulas are listed in Section III of supplementary material. Our data comes from Tushare,¹ a free and open-source Python financial data interface package. Its data comes from Yahoo Finance and major Chinese stock exchanges.

¹<https://tushare.pro/>

B. Comparison Methods

Since machine learning has gained so much popularity, current research on TP forecasting has focused on leveraging statistical machine learning methods [33] and more complex neural network methods [23]. In order to make our experiment more convincing, we have also added some excellent machine learning methods commonly used for predicting stock price trends, such as long short-term memory (LSTM) [34], TCNs [20], and attention-based methods [19], [36], which use price data or technical indicators as features, and have attained competitive performance. In a word, to verify the effectiveness of our proposed method of robustly and convergently utilizing chart information to predict TPs, we compare it with the following baselines:

- *SVM* [33]: the first baseline algorithm is Support Vector Machine.
- *LSTM* [34]: the second baseline uses a simple LSTM network to forecast stock price trends based on price data.
- *TCN* [20]: temporal convolution network (TCN), the third baseline, involves multilayer convolution on the timestamp dimension.
- *FDDR* [15]: fuzzy deep direct reinforcement (FDDR), the fourth baseline, combines a recurrent deep neural network and reinforcement learning for real-time financial signal representation and trading.
- *DARNN* [36]: the fifth basic technique, the dual-stage attention-based recurrent neural network (DARNN), outperforms the LSTM and attention-based LSTM in terms of time-series prediction performance by selecting relevant source series as well as temporal variables.
- *CA-FSCN* [19]: the sixth baseline model, known as CA-FSCN (Cross-Attention Stabilized Fully Convolutional Neural Network), employs a fully convolutional network (FCN) with a dual-stage attention mechanism. This cross-attention mechanism operates on both the variable and timestamp dimensions, allowing the model to select relevant variables at each time step. The effectiveness of this approach has been demonstrated in [19].
- *CNN-LSTM-ResNet model (CNN-LSTM)* [23]: the latest method, referred to as the sixth baseline model, utilizes a hybrid Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) architecture to predict TPs in stock prices.

To demonstrate the efficacy of our framework in predicting stock movement using actual data, we employ technical indicator data as input in all baseline techniques, which all maintain consistency with the original model.

C. Parameter Setting

To prevent data snooping, we adhere to a strict division of the training, validation, and test sets based on the timestamp of the sample data. Specifically, data spanning from January 2010 to December 2019 is allocated to both the training and validation sets, while the entirety of the data from the year 2020 is reserved as the test set. To leverage historical information effectively, we randomly assign 80% of the samples to the

TABLE I
SUMMARY STATISTICS OF THE SZ-50 AND CSI-300 INDICES FOR THE TRAINING, VALIDATION AND TEST DATASETS

Dataset		Training		Validation		Testing	
		(%)	#Samples	(%)	#Samples	(%)	#Samples
SZ-50	<i>OPs</i>	47.20	128,650	46.77	14,137	47.86	14,837
	<i>TPs</i>	52.80		53.23		52.14	
CSI-300	<i>OPs</i>	50.04	814,893	49.36	82,771	48.22	83,512
	<i>TPs</i>	49.96		50.64		51.78	

TABLE II
RESULTS: THE BASELINES AND FRAMEWORK WE PROPOSED

	SZ-50					CSI-300				
	Acc	Pre_1	Pre_0	F1_1	F1_0	Acc	Pre_1	Pre_0	F1_1	F1_0
SVM	0.5639	0.5289	0.5979	0.5446	0.5817	0.5712	0.5368	0.6036	0.5489	0.5914
LSTM	0.5835	0.5500	0.6146	0.5597	0.6049	0.5713	0.5368	0.6044	0.5504	0.5904
TCN	0.5899	0.5574	0.6192	0.5634	0.6133	0.5726	0.5382	0.6053	0.5510	0.5922
DARNN	0.6217	0.5898	0.6513	0.5999	0.6413	0.5794	0.5452	0.6121	0.5584	0.5986
CNN-LSTM	0.6570	0.6248	0.6875	0.6395	0.6728	0.6249	0.5925	0.6553	0.6046	0.6432
CA-FSCN	0.6924	0.6638	0.7188	0.6742	0.7087	0.6426	0.6112	0.6718	0.6225	0.6607
Our Method	0.7416	0.7140	0.7672	0.7269	0.7547	0.7087	0.6800	0.7354	0.6921	0.7236

All models predict TP labels of the following time step.

training set, with the remaining 20% designated as the validation set during the training process. The statistical characteristics of the SZ-50 and CSI-300 indices datasets can be found in Table I. It is worth noting that, thanks to our improved PIP method, the ratio between TPs and OPs has been balanced. For actual trading purposes, we employ the closing price for graph transformation.

In this paper, we employ a grid search to identify the optimal hyperparameters that yield the best performance across all methods. Specifically, for the MPCNet model, it consists of (2, 3, 4, 5) layers with a hidden feature dimension of (8, 16, 32, 64). The final hyperparameter settings include a learning rate of (0.1, 0.01, 0.001) and a batch size of 128. The cost $d_{uu}^{t_a t_a}$ associated with staying in the same node u within layer t_a , and the cost $d_{uu}^{t_a t_b}$ of transitioning from node u in layer t_a to node u in layer t_b , are both set to (0.1, 0.01, 0.001). Furthermore, the embedding dimension d_c in E_ϕ is set to (8, 16, 32, 64) for all price graphs.

In our experiments, for all datasets, the CNN-based model (f_{θ_j} , where $j = 1, 2$) within the Multipersistence Euler-Poincaré Surface Representation Learning (MEPSRL) module consists of 2 CNN layers. The filter size, kernel size, and stride are set to 8, 2, and 2, respectively. The size of global average pooling and global max pooling is set to 3×3 . The optimal output dimensions for the spatial graph convolutional layer, supragraph diffusion convolutional layer, and spatio-temporal feature transformation are set to 50%, 30%, and 20%, respectively, across all datasets.

All the aforementioned baselines utilize the same set of common parameters, including a regularization weight decay of 0.00005 and a batch size of 128, as empirically determined.

V. RESULTS

Along with seven state-of-the-art baselines on the CSI-300 and SZ-50 index components in 2020, Table II shows the performance comparison to confirm the overall effectiveness

of our framework on actual stock data. Table II shows the overall performance in terms of accuracy, precision, and F1 score, whose formulas are listed in Section III of supplementary material. Among them, precision and F1 score are divided into the results of predicting the TPs and OPs, corresponding to "_1" and "_0" respectively. As long as we make appropriate trading decisions with caution in each transaction, we can benefit from the characteristics of stock trading. Our MPCNet consistently outperforms the baseline model on all two datasets, highlighting the effectiveness of MPCNet in TP prediction tasks.

LSTM and TCN, which consider temporal dependencies, perform better compared with traditional statistical machine learning algorithms such as SVM. The performance of the FDDR method is similar to that of methods such as LSTM and TCN. One possible reason is that the original prediction target of the FDDR method was the movement of minute-level closing price data, while our data is daily data, and the prediction target is TPs, not price movement. DARNN and CA-FSCN utilize self-attention modules, resulting in a significant improvement in prediction performance. CA-FSCN utilizes attention mechanisms in both timing and features, resulting in better performance compared with DARNN. CNN-LSTM performs well by using CNN to extract technical chart information for TP prediction tasks with targeted techniques. However, due to the inability to ensure convergence and stability when extracting technical chart information through CNN, the final effect is not as good as CA-FSCN.

The effectiveness of our method on two datasets can demonstrate the advantages of MPCNet. Please note that for most models, the prediction performance on SZ-50 is better than CSI-300. One possible reason is that the number of stocks in SZ-50 is smaller than CSI-300, and most of the stocks in SZ-50 are leading stocks, making it more difficult for capital to manipulate their stock prices. That is to say, compared to CSI-300, SZ-50 has more representative and useful chart information.

TABLE III
RESULTS OF THE ABLATION ANALYSIS INCLUDE THE ACCURACY OF EACH MODEL VARIANT, PRECISION, AND F1 SCORE

	SZ-50					CSI-300				
	Acc	Pre_1	Pre_0	F1_1	F1_0	Acc	Pre_1	Pre_0	F1_1	F1_0
MPCNet-1	0.5668	0.5318	0.6012	0.5486	0.5836	0.5657	0.5310	0.5987	0.5441	0.5853
MPCNet-2	0.5935	0.5597	0.6257	0.5729	0.6123	0.5925	0.5601	0.6220	0.5667	0.6155
MPCNet-3	0.6520	0.6215	0.6803	0.6317	0.6703	0.6045	0.5719	0.6347	0.5816	0.6249
MPCNet-4	0.6817	0.6512	0.7102	0.6643	0.6973	0.6655	0.6375	0.6906	0.6432	0.6851
MPCNet-5	0.6480	0.6207	0.6720	0.6236	0.6693	0.6221	0.6706	0.6735	0.6239	0.6718
MPCNet	0.7416	0.7140	0.7672	0.7269	0.7547	0.7087	0.6800	0.7354	0.6921	0.7236

Our method has significant advantages in utilizing representative chart information to predict key point, namely TPs, and therefore performs better on SZ-50. At the same time, we found that MPCNet has strong competitiveness even compared to a baseline that integrates partial topology information (i.e., CNN-LSTM), which means that multipersistence representations with MEPS can better encode temporal conditional information compared to CNN-LSTM based on price formed images. Furthermore, all methods are more accurate at predicting OPs than TPs because the arbitrage space at the TP is larger. According to the efficient market hypothesis, more market participants hope to predict or utilize the TP, making the TP more difficult to predict.

Additionally, more experiments of SCI-300 and SZ-50 are conducted on 2021 and 2022 and we also test our model in USA market. The results are listed in Section I of supplementary material.

VI. DISCUSSION

A. Ablation

We compared our MPCNet with five ablation variants, namely (i) MPCNet without PIP module (w/o PIP module), (ii) MPCNet without spatial graph convolutional layer (w/o GCNSpa module), (iii) MPCNet without FT (w/o FT), (iv) MPCNet without MEPSRL module (w/o MEPSRL module), and (v) MPCNet without the self-attention layer (w/o self-attention module). The results are listed together with the precision results in Table III.

We observed that the prediction performance of MPCNet-1 decreased the most compared to our complete version, as the scale of the charts is variable and different scales of charts are mixed. The original price sequence without PIP contains a lot of noise and is difficult to distinguish. Compared with the complete MPCNet, the decrease of MPCNet-2 and MPCNet-3 on SZ-50 is basically the same, while on CSI-300, the decrease of MPCNet-2 is significantly greater than that of MPCNet-3. First, the absence of these two modules will have a certain negative impact on the selection of charts. One possible reason for the difference between the two datasets is that SZ-50 has a larger number of constituent stocks and longer investment cycles for investors. Therefore, the historical chart information represented by the FT module has a greater impact on prediction, and the loss of this module has a greater impact on the constituent stocks of this index. We noticed that MPCNet-4 has the smallest difference compared to our method, as chart similarity has already ensured robustness and has relatively good extraction

ability for chart information. Our method has a greater lead on SZ-50 compared to MPCNet-4, which also indicates that our method further enhances the ability to extract representative information from charts by ensuring convergence on the basis of robustness, as the constituent stocks of SZ-50 are all leading stocks with more representative charts, which is more conducive to our method. We observed that MPCNet-5 did not perform as well as MPCNet on both dataset. The possible reason is that the meaning represented by the chart changes over time, so it is necessary to capture spatio-temporal correlations among time series and obtain the hidden state of nodes at a future timestamp through self-attention layers.

B. The Robustness of Our Framework in Similarity Measurement

To verify the robustness of our proposed framework, we calculate the similarity of the $\mathbb{EP}$ matrix obtained through PIP, VG, and multipersistence calculations. The calculation example is shown in the Fig. 1. In this study, we compare the performance of the Pearson correlation coefficient, DTW distance [37], and DTW distance without normalization [45] under various levels of small variations in time scaling, shift, amplitude scaling, rotation, and additive Gaussian noise, as previously explored in [24]. Our main focus lies in assessing the insensitivity of these similarity methods to small data variations. The time-series variations are precisely defined as follows:

- Amplitude scaling*: $X_{AS}(t) = \beta * X(t)$
- Amplitude shift*: $X_{At}(t) = X(t) + \beta$
- Time scaling*: $X_{Ts}(t) = X(\beta * t)$
- Time shift*: $X_{Tt}(t) = X(t + \beta)$
- Rotation*: $X_B(t) = \text{rotate}(X(t), \beta)$
- WGN added*: $X_{WGN}(t) = X(t) + Z_i \sim \mathcal{N}(X_i, \beta)$

The corresponding diagrams depicting different variations are illustrated in Fig. 1 with corresponding serial numbers. In these diagrams, β represents a constant indicating the scaling, shift magnitude or the rotation angle, and WGN refers to white Gaussian noise. To assess the insensitivity of the similarity methods to small data variations, we perform 20-200 small incremental changes on the β values, with their initial values determined through experimentation. For each variation type (from (a) to (f)), the horizontal ordinate of the figures in Fig. 5 indicates these 20-200 degrees of variation, while the vertical ordinate represents the similarity between the variations and the original time series. To calculate the similarity, we select the stock closing

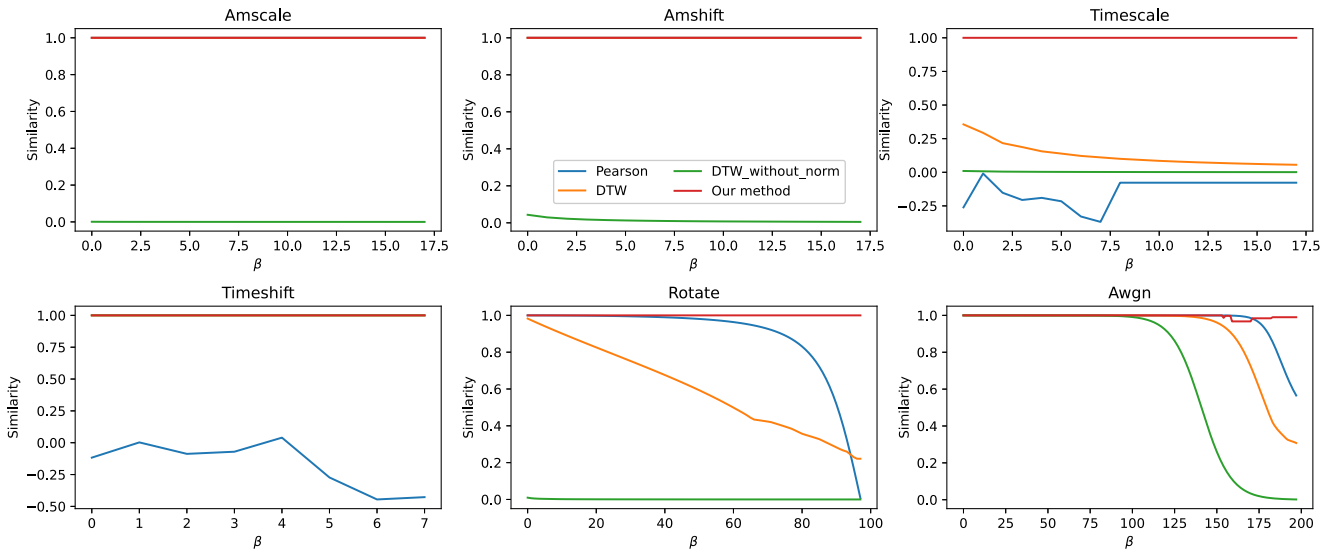


Fig. 5. Comparison of different similarity methods on different variations.

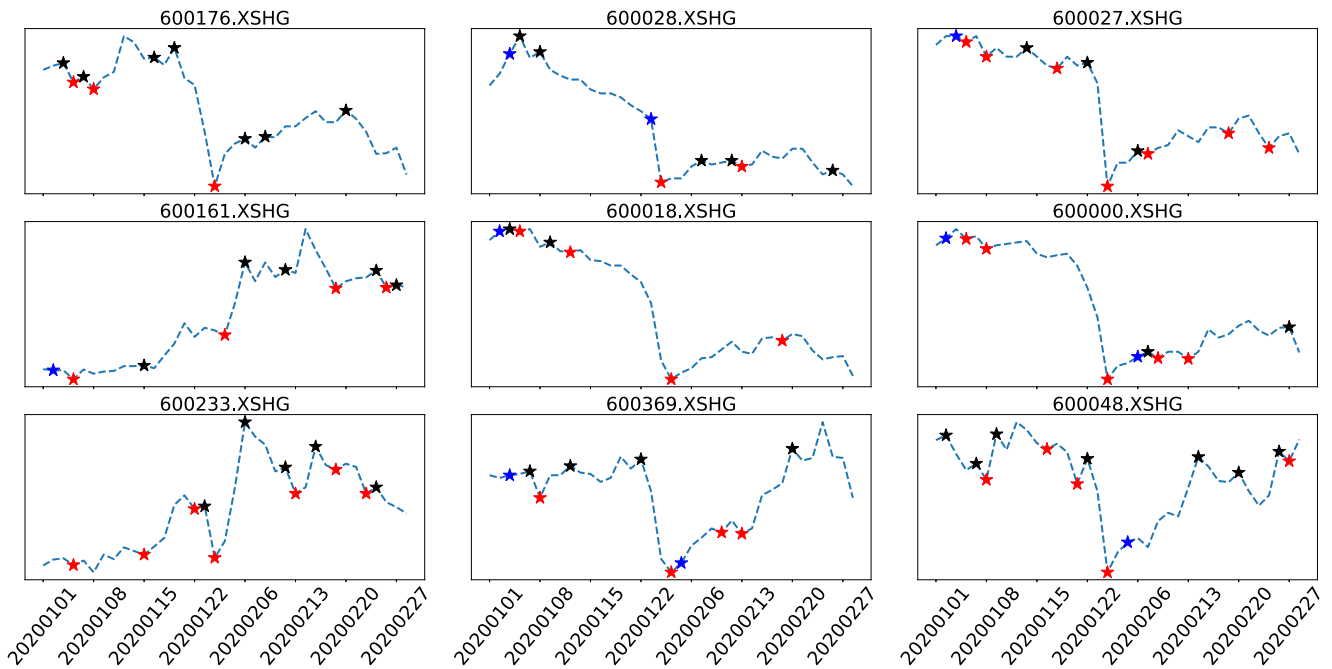


Fig. 6. The visualization of predicted TPs.

price in the CSI-300 from 2010 to 2020 and consider 20 trading days (almost one month) as an interval for intercepting the price series. The similarity is then calculated with the original price series under different degrees of variation. The average value of all intercepted price series under a certain degree of variation is presented in Fig. 5. A higher similarity value indicates that the variation has less impact on the similarity method.

As shown in the Fig. 5, the results indicate that our similarity method is the most robust. When the amplitude of price changes or shift, our method maintains stability under varying degrees of magnitude disturbance. When the time scale changes or moves,

the Pearson correlation coefficient method performs very poorly, while our method and DTW method both perform very well and may even be unaffected. Especially, under high levels of WGN changes, the similarity of our method remains at the highest level and is stable.

C. The Visualization of Chart Interpretability

Fig. 6 shows the TP obtained from the proposed model among 9 randomly selected stocks. We mark the predicted TPs with three colors. The TP at the valley is marked in red, the TP at the

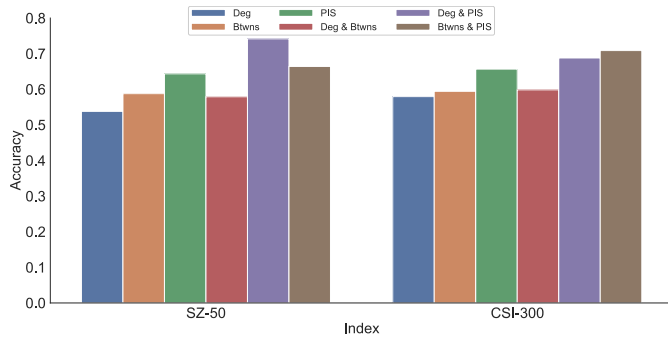


Fig. 7. The contribution of single persistence versus multipersistence.

peak is marked in black, and the wrong predicted TP (i.e., OP is predicted as TP) is marked in blue. The red mark is a buy signal, the black mark is a sell signal.

For example, in stock 600233, our method accurately identifies most of the important peak regions and provides profitable selling signals, which is crucial for strategic risk control and allows investors to avoid risks in a timely manner to ensure their own returns. In stock 600028, although there were two prediction errors, our method predicts most of the volatile TPs, which is very beneficial for strategic returns, as some large TPs are often the key factors affecting strategic returns. In the 600027 stock market, our method finds very uniform and meaningful TP signals, making it convenient for investors to trade or improve their strategies. The above indicates that our proposed model can locate TP and provide investment advice.

D. The Contribution of Single Persistence Versus Multipersistence

We incorporate sub-level filtering utilizing node degree (Deg), betweenness (Btwns), and PIP importance score (PIS). The first two are selected from strict Graph properties, and the last one is the TP prediction domain function. Fig. 7 illustrates that multipersistence generally outperforms single persistence in terms of prediction accuracy and variability.

These findings support our hypothesis that multipersistence has the ability to (i) capture hidden chart information in high-dimensional time-varying objects that cannot be accessed with a single persistence, and (ii) generate more reliable feature maps in dynamic scenarios. At the same time, both Btwns and Deg belong to graph properties, and the guarantee for MP to produce good results is unrelated filters of different fields as we mentioned in the previous theoretical proof. Therefore, our experimental results here comply with the requirements of MP. We also observed that the combination of PIS and graph features performs better, but the combination filter that achieves the best results on different datasets is different. This may be because the stock volatility in CSI-300 is greater, so the key points and edge information in the chart are more important, accordingly PIS and Btwns can better assist in prediction.

E. Trading Simulation

The same trading technique guarantees that the relative order on the basis of the model return stays constant, despite the

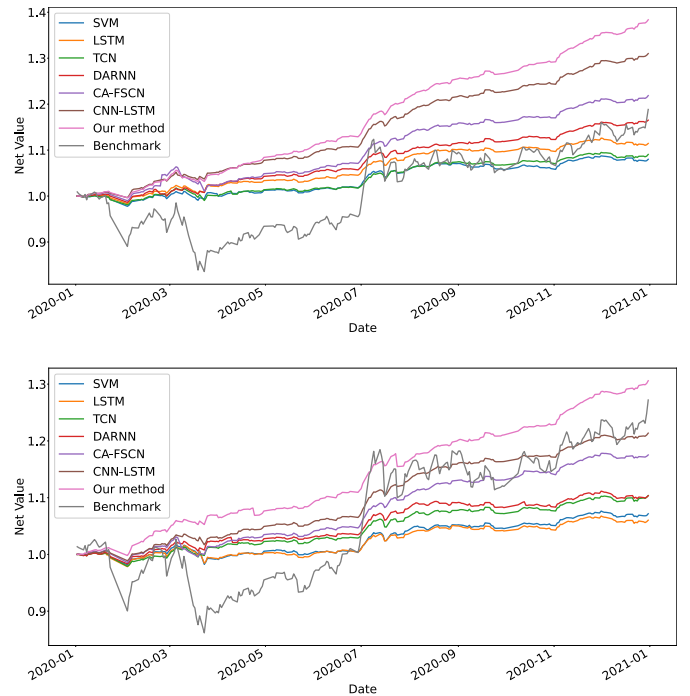


Fig. 8. The backtest results. **Upper panel:** SZ-50; **Lower panel:** CSI-300.

transaction cost influencing overall profit. Holding each stock in the same proportion allows us to determine the average return on the stocks that make up the two indices as benchmarks, demonstrating the general direction of the market. In the Chinese stock market, one can only make long trades on stocks and cannot make short trades. Our trading strategy designed for the predicted target is as follows: once our model predicts that a certain stock will have a TP at the next time point and the closing price of the day will decrease compared to the previous day, we will buy the stock at the close of the day. However, it should be noted that if our holdings of stocks have already been established, we will not engage in any trading. When our model predicts that the next time point of a stock is a TP and the closing price of the day rises compared to the previous day, if we have a long position in the stock, we will sell it at the opening of the day. MACD, another technical indicator, is also used to reduce volatility risk. If the MACD value of the stock is below 0, execute the long-short program. In Fig. 8, the net-value curves of different models are shown. During the market volatility in the first half of 2020, our method still achieved the maximum profit result among all baselines. Meanwhile, on the CSI-300 index, only our method outperformed the index.

VII. CONCLUSION

This study proposed a novel hybrid model entitled MPCNet to forecast TPs. In order to better utilize chart information, whose crucial judgments are TPs, we use robust chart similarity algorithms such as the PIP algorithm and the VG algorithm to convert price sequences into graph structures. Meanwhile, we investigate the utilization of multipersistence to improve chart information representation techniques and ensure convergence by providing theoretical proof. Meanwhile an enhanced PIP and

balancing algorithm were employed to generate TPs/OPs. In contrast to existing baseline models, the findings show that our proposed model achieve higher accuracy and robustness during the out-of-sample test periods for both the CSI-300 stocks and the SZ-50 stocks. In other words, the robustness and accuracy of our suggested model can be improved by all the suggested modules, including chart similarity and multipersistence. The backtest results also show that the suggested MPCNet model outperformed other comparable models in the CSI-300 stocks and SZ-50 stocks for earnings.

Although this study offers insightful information, it still has some limitations. The impact among different stocks is crucial, and there may be connections and interactions between the technical charts of various stocks. Meanwhile, our approach is orthogonal and complementary to more types of information such as trading volume, news, and economic indices. Therefore, our future research will explore the information and collaboration between technical charts of various stocks and more data. By combining the input characteristics from this study with environmental aspects and fundamental indicator features, such as oil, gold, bitcoin, and interest rates, the suggested model may be more useful. Additionally, investigating the asymptotic distributional characteristics of point-wise multipersistence invariants and combining multipersistence with zigzag persistence are two excellent suggestions for future research that might create more informative graphs.

REFERENCES

- [1] D. F. Ahelegbey, M. Billio, and R. Casarin, "Modeling turning points in the global equity market," *Econometrics Statist.*, vol. 30, 2021, pp. 60–75.
- [2] T. Belkhouja, Y. Yan, and J. R. Doppa, "Dynamic time warping based adversarial framework for time-series domain," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7353–7366, Jun. 2023.
- [3] G. Beltramo, P. Skraba, R. Andreeva, R. Sarkar, Y. Giarratano, and M. O. Bernabeu, "Euler characteristic surfaces," *Found. Data Sci.*, vol. 4, no. 4, pp. 505–536, 2022.
- [4] A. Carbone, G. Castelli, and H. E. Stanley, "Time-dependent hurst exponent in financial time series," *Physica A: Statist. Mechan. Appl.*, vol. 344, no. 1/2, pp. 267–271, 2004.
- [5] G. Carlsson and R. B. Gabrielsson, "Topological approaches to deep learning," in *Proc. Topological Data Anal.: Abel Symp.*, Springer, 2020, pp. 119–146.
- [6] M. Carriere and A. Blumberg, "Multiparameter persistence image for topological machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 22432–22444.
- [7] M. Carrière, F. Chazal, Y. Ike, T. Lacombe, M. Royer, and Y. Umeda, "PersLay: A neural network layer for persistence diagrams and new graph topological signatures," in *Proc. Int. Conf. Artif. Intell. Statist.*, PMLR, 2020, pp. 2786–2796.
- [8] R. Cervello-Royo, F. Guijarro, and K. Michniuk, "Stock market trading rule based on pattern recognition and technical analysis: Forecasting the djia index with intraday data," *Expert Syst. Appl.*, vol. 42, no. 14, pp. 5963–5975, 2015.
- [9] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, and Z. Jia, "Freeway performance measurement system: Mining loop detector data," *Transp. Res. Rec.*, vol. 1748, no. 1, pp. 96–102, 2001.
- [10] J. Chen, S. Yang, D. Zhang, and Y. A. Nanekharan, "A turning point prediction method of stock price based on RVFL-GMDH and chaotic time series analysis," *Knowl. Inf. Syst.*, vol. 63, no. 10, pp. 2693–2718, 2021.
- [11] Y. Chen, I. Segovia-Dominguez, B. Coskunuzer, and Y. Gel, "TAMP-S2GCNets: Coupling time-aware multipersistence knowledge representation with spatio-supra graph convolutional networks for time-series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–28.
- [12] D. Chin, J. Geweke, and P. Miller, "Predicting turning points. Federal reserve bank of minneapolis," Research Department Staff Report 267, 2000.
- [13] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer, "Stability of persistence diagrams," in *Proc. 21st Annu. Symp. Comput. Geometry*, 2005, pp. 263–271.
- [14] R. Corbet, U. Fugacci, M. Kerber, C. Landi, and B. Wang, "A kernel for multi-parameter persistent homology," *Comput. Graph.: X*, vol. 2, 2019, Art. no. 100005.
- [15] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 653–664, Mar. 2017.
- [16] H. Edelsbrunner and J. L. Harer, *Computational Topology: An Introduction*. Providence, RI, USA: AMS, 2022.
- [17] D. Grech and Z. Mazur, "Can one make any crash prediction in finance using the local hurst exponent idea?," *Physica A: Statist. Mechan. Appl.*, vol. 336, no. 1/2, pp. 133–145, 2004.
- [18] H. Guo, L. Wang, X. Liu, and W. Pedrycz, "Information granulation-based fuzzy clustering of time series," *IEEE Trans. Cybern.*, vol. 51, no. 12, pp. 6253–6261, Dec. 2021.
- [19] Y. Hao and H. Cao, "A new attention mechanism to classify multi-variate time series," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 1999–2005.
- [20] P. Hewage et al., "Temporal convolutional neural (TCN) network for an effective weather forecasting using time-series data from the local weather station," *Soft Comput.*, vol. 24, no. 21, pp. 16453–16482, 2020.
- [21] C. Ji, M. Du, Y. Hu, S. Liu, L. Pan, and X. Zheng, "Time series classification based on temporal features," *Appl. Soft Comput.*, vol. 128, 2022, Art. no. 109494.
- [22] P. Juszczuk, J. Kozak, and K. Kania, "Using similarity measures in prediction of changes in financial market stream data—experimental approach," *Data Knowl. Eng.*, vol. 125, 2020, Art. no. 101782.
- [23] P. Khodaei, A. Esfahanipour, and H. M. Taheri, "Forecasting turning points in stock price by applying a novel hybrid CNN-LSTM-resnet model fed by 2D segmented images," *Eng. Appl. Artif. Intell.*, vol. 116, 2022, Art. no. 105464.
- [24] A. Kianimajd et al., "Comparison of different methods of measuring similarity in physiologic time series," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 11005–11010, 2017.
- [25] L. Kristoufek, "Local scaling properties and market turning points at prague stock exchange," *Acta Physica Polonica B*, vol. 41, no. 6, 2010, Art. no. 1223.
- [26] L. Lacasa and W. Just, "Visibility graphs and symbolic dynamics," *Physica D: Nonlinear Phenomena*, vol. 374, pp. 35–44, 2018.
- [27] L. Lacasa, B. Luque, F. Ballesteros, J. Luque, and J. C. Nuno, "From time series to complex networks: The visibility graph," in *Proc. Nat. Acad. Sci.*, vol. 105, no. 13, pp. 4972–4975, 2008.
- [28] W. Leigh, N. Modani, and R. Hightower, "A computational implementation of stock charting: Abrupt volume increase as signal for movement in new york stock exchange composite index," *Decis. Support Syst.*, vol. 37, no. 4, pp. 515–530, 2004.
- [29] B. Li et al., "DiffFormer: Multi-resolutional differencing transformer with dynamic ranging for time series analysis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13586–13598, Nov. 2023.
- [30] S. Li, J. Wu, X. Jiang, and K. Xu, "Chart GCN: Learning chart information with a graph convolutional network for stock movement prediction," *Knowl.-Based Syst.*, vol. 248, 2022, Art. no. 108842.
- [31] A. W. Lo, H. Mamaysky, and J. Wang, *Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation*. New York, NY, USA: Social Science Electronic Publishing, 2000.
- [32] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Commun. ACM*, vol. 22, no. 10, pp. 560–570, 1979.
- [33] L. Luo, S. You, Y. Xu, and H. Peng, "Improving the integration of piece wise linear representation and weighted support vector machine for stock trading signal prediction," *Appl. Soft Comput.*, vol. 56, pp. 199–216, 2017.
- [34] D. MQ Nelson, A. CM Pereira, and R. A. de Oliveira, "Stock market's price movement prediction with LSTM neural networks," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 1419–1426.
- [35] D. Oner, A. Garin, M. Kozłowski, K. Hess, and P. Fua, "Persistent homology with improved locality information for more effective delineation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 10588–10595, Aug. 2023.

- [36] Y. Qin et al., "A dual-stage attention-based recurrent neural network for time series prediction," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 2627–2633.
- [37] P. Senin, "Dynamic time warping algorithm review," *Inf. Comput. Sci. Dept. Univ. Hawaii Manoa Honolulu, USA*, vol. 855, no. 1/23, 2008, Art. no. 40.
- [38] O. B. Sezer and A. M. Ozbayoglu, "Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach," *Appl. Soft Comput.*, vol. 70, pp. 525–538, 2018.
- [39] H. J. Stock and M. W. Watson, "Estimating turning points using large data sets," *J. Econometrics*, vol. 178, pp. 368–381, 2014.
- [40] J. Su, D. V. Vargas, and K. Sakurai, "One pixel attack for fooling deep neural networks," *IEEE Trans. Evol. Comput.*, vol. 23, no. 5, pp. 828–841, Oct. 2019.
- [41] H. Tang, P. Dong, and Y. Shi, "A new approach of integrating piecewise linear representation and weighted support vector machine for forecasting stock turning points," *Appl. Soft Comput.*, vol. 78, pp. 685–696, 2019.
- [42] M. Tarnopolski, "Analytical representation of Gaussian processes in the A-T plane," *Phys. Rev. E*, vol. 100, no. 6, 2019, Art. no. 62144.
- [43] A. L. Thomas, "Invariants and metrics for multiparameter persistent homology," PhD Thesis, Duke University, 2019.
- [44] P. Tsinaslanidis and F. Guijarro, "What makes trading strategies based on chart pattern recognition profitable?," *Expert Syst.*, vol. 38, no. 5, 2021, Art. no. e12596.
- [45] P. E. Tsinaslanidis, "Subsequence dynamic time warping for charting: Bullish and bearish class predictions for NYSE stocks," *Expert Syst. Appl.*, vol. 94, pp. 193–204, 2018.
- [46] P. E. Tsinaslanidis and D. Kugiumtzis, "A prediction scheme using perceptually important points and dynamic time warping," *Expert Syst. Appl.*, vol. 41, no. 15, pp. 6848–6860, 2014.
- [47] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [48] O. Vipond, "Multiparameter persistence landscapes," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 2262–2299, 2020.
- [49] N. Wang, D. Li, and Q. Wang, "Visibility graph analysis on quarterly macroeconomic series of China based on complex network theory," *Physica A: Statist. Mechan. Appl.*, vol. 391, no. 24, pp. 6543–6555, 2012.
- [50] Y. Wang, Y. Qu, and Z. Chen, "Review of graph construction and graph learning in stock price prediction," *Procedia Comput. Sci.*, vol. 214, pp. 771–778, 2022.
- [51] W. E. Wecker, "Predicting the turning points of a time series," *J. Bus.*, vol. 52, pp. 35–50, 1979.
- [52] J. Wu, K. Xu, X. Chen, S. Li, and J. Zhao, "Price graphs: Utilizing the structural information of financial time series for stock prediction," *Inf. Sci.*, vol. 588, pp. 405–424, 2022.
- [53] N. Zhang and S. Sun, "Multiview unsupervised shapelet learning for multivariate time series clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4981–4996, Apr. 2023.



Shangzhe Li received the BE degree from Beihang University, Beijing, China, in 2018. He is currently working toward the PhD degree with the School of Mathematics Science, Beihang University, Beijing, China. His research interests include graph learning and time series analysis.



Yingke Liu received the BE degree from Southwest University, Chongqing, China, in 2018. She is currently working toward the PhD degree with the School of Statistics and Mathematics, Central University of Finance and Economics, Beijing, China. Her research interests include climate finance, energy economy and financial time series analysis.



Xueyuan Chen received the BE degree from Beihang University, Beijing, China, in 2016. He is currently working toward the PhD degree with the Department of Computer Science and Technology, Beihang University, Beijing, China. His research interests include spatial-temporal graph neural networks and intelligent transportation.



Junran Wu received the BE degree from Beihang University, Beijing, China, in 2016. He is currently working toward the PhD degree with the Department of Computer Science and Technology, Beihang University, Beijing, China. His research interests include graph learning and natural language processing.



Ke Xu received the BE, ME and PhD degrees from Beihang University, in 1993, 1996, and 2000, respectively. He is a professor with Beihang University, China. His research interests include algorithm and complexity, data mining and complex networks.